WEJ LAB

Regularization in Regression

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-28

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Explain the bias-variance tradeoff
- Describe ridge (L2), lasso (L1), and elastic net penalties
- Explain why scaling matters and how glmnet handles it
- Fit and tune penalized models with glmnet and cross-validation
- Choose between λ_{min} and λ_{1se}
- Apply penalized logistic regression to NHANES diabetes data

Bias-Variance tradeoff

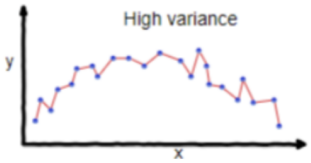
- **Bias–variance tradeoff:** we can reduce the variance of estimates by accepting some bias
- **Bias–variance dilemma:** we can't minimize both at once, and both limit how well a model generalizes beyond its training set

(Source: Wikipedia)

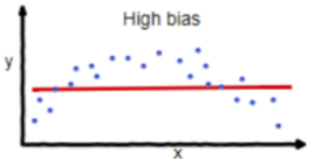
Bias-Variance tradeoff

- **Bias:** error from wrong assumptions; misses real relationships (**underfitting**)
- **Variance:** error from sensitivity to the training set; models the noise (**overfitting**)

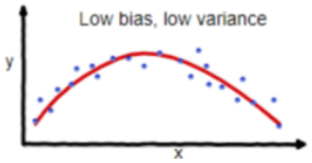
Bias-Variance tradeoff



overfitting

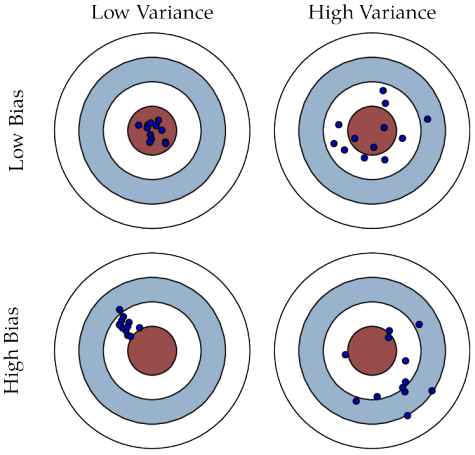


underfitting



Good balance

Bias-Variance tradeoff



Bias-Variance tradeoff

A central problem in supervised learning. We want a model that:

- captures the patterns in the training data, **and**
- generalizes to new data

Usually we can't have both:

- High variance: fits training data well, but overfits noise
- High bias: simpler, but underfits real patterns

(Source: Wikipedia)

Regularization in Machine Learning

- With small samples or noisy data, coefficient estimates can be inaccurate (overfitting)
- **Regularization** restricts or shrinks estimates toward zero:
  - reduces the impact of unimportant features
  - adds a little bias to cut variance and overfitting
  - improves performance on new data

See *An Introduction to Statistical Learning* (James et al., 2nd ed.), Ch. 6: https://www.statlearning.com

Regularization in Machine Learning

Examples:

- Regression: penalize coefficients for parsimonious models (variable selection)
- Neural networks: constrain the weights
- Random forests: limit tree depth

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Ridge regression

Ridge regression **shrinks** coefficients by penalizing their size:

$$\hat{\beta}^{ridge} = \arg\min_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\},$$

where $\lambda \geq 0$ controls the shrinkage: larger λ, more shrinkage toward 0.

Ridge regression

In matrix form:

$$\hat{\beta}^{ridge} = \arg\min_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \right\}.$$

with closed-form solution:

$$\hat{\beta}^{ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^T \mathbf{y}$$

Ridge regression

The ridge penalty $\lambda\beta^T\beta$ is an **L2 penalty** (squared size of the coefficients). SVMs use it too.

- Handles multicollinearity: shrinks coefficients but keeps all variables
- Does **not** select variables. For that, we use an **L1 penalty**.

Ridge regression: a Bayesian perspective

- Ridge $=$ posterior mode under a **Normal** prior on β with mean zero; $\lambda = \sigma^2/\tau^2$ is inversely related to the prior variance τ^2
- Lasso (next) $=$ posterior mode under a **Laplace** (double-exponential) prior, which has a sharp peak at zero

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Lasso regression

Lasso (Least Absolute Shrinkage and Selection Operator) shrinks coefficients with an **L1** penalty:

$$\hat{\beta}^{lasso} = \arg\min_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right\},$$

where $\lambda \geq 0$ controls the shrinkage: larger λ, more shrinkage toward 0.

Lasso regression

- Same as ridge, with L2 $\sum_{j=1}^{p} \beta_j^2$ replaced by L1 $\sum_{j=1}^{p} |\beta_j|$
- No closed-form solution (nonlinear in the y_i)
- A large enough λ sets some coefficients **exactly to zero**: the lasso does **variable selection**

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Lasso vs Ridge regression

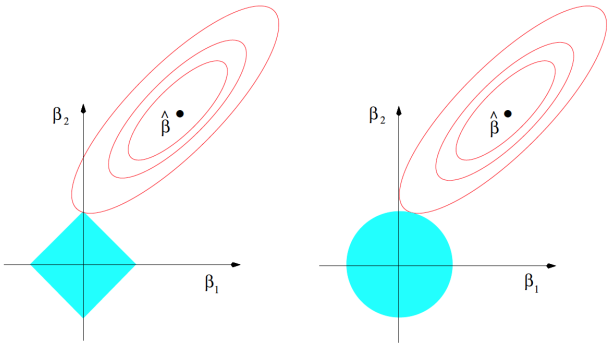


FIGURE 3.11. *Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions* $|\beta_1| + |\beta_2| \leq t$ *and* $\beta_1^2 + \beta_2^2 \leq t^2$*, respectively, while the red ellipses are the contours of the least squares error function.*

(Elements of Statistical Learning, Hastie, Tibshirani, Friedman, by Springer)

Lasso vs Ridge regression

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

(https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning)

Elastic net regularization

Ridge or lasso? Why choose! The **elastic net** minimizes:

$$\hat{\beta}^{\text{elastic net}} = \arg\min_{\beta} \left\{ \sum_{i=1}^{N} (y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda_1 \sum_{j=1}^{p} |\beta_j| + \lambda_2 \sum_{j=1}^{p} \beta_j^2 \right\},$$

for $\lambda_1 \geq 0$, $\lambda_2 \geq 0$.

- The quadratic term makes the loss strongly convex (unique minimum)
- Special cases: ridge ($\lambda_1 = 0$), lasso ($\lambda_2 = 0$), OLS (both 0)
- With **correlated** predictors, the lasso tends to keep one and drop the rest; the elastic net keeps the group together

Regression Regularization in R: glmnet

Regularization in R with **glmnet**:

```
install.packages("glmnet")
```

Regression Regularization in R: glmnet

For least squares (the default), glmnet minimizes:

$$\arg\min_{\beta_0,\beta} \left\{ \frac{1}{2N} \sum_{i=1}^{N} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \left(\alpha \sum_{j=1}^{p} |\beta_j| + \frac{1-\alpha}{2} \sum_{j=1}^{p} \beta_j^2 \right) \right\}$$

- λ: overall penalty strength; α: mix of lasso ($\alpha = 1$, the default) and ridge ($\alpha = 0$)
- The intercept β_0 is **not** penalized

Scaling matters

- The penalty treats all coefficients the same, but a coefficient's size depends on its predictor's **units**
- So predictors must be on a common scale
- glmnet **standardizes** predictors internally by default (`standardize = TRUE`), then reports coefficients on the original scale

Regression Regularization in R: glmnet

The package's quick start example:

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
data(QuickStartExample)
x <- QuickStartExample$x
y <- QuickStartExample$y
```

Regression Regularization in R: glmnet

The most basic call:

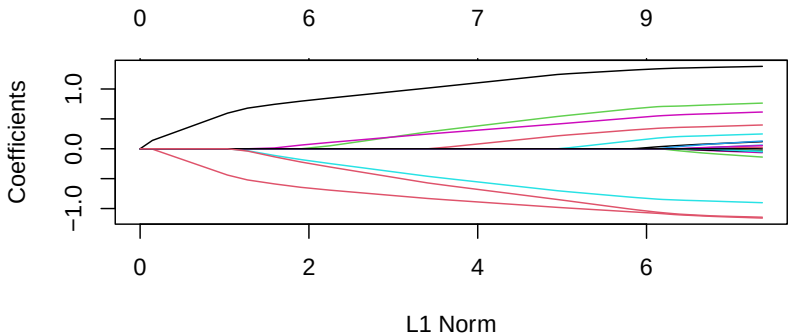
```
fit <- glmnet(x, y)
```

`fit` holds the full fitted model. Use `plot`, `print`, `coef`, and `predict` rather than extracting components directly.

Regression Regularization in R: glmnet

Coefficient paths:

```
plot(fit)
```



WEJ LAB

RUTGERS HEALTH
Center for Data Science

Regression Regularization in R: glmnet

`print` summarizes the path at each step:

```
print(fit)
```

```
##
## Call:  glmnet(x = x, y = y)
##
##    Df  %Dev  Lambda
## 1   0  0.00 1.63100
## 2   2  5.53 1.48600
## 3   2 14.59 1.35400
## 4   2 22.11 1.23400
## 5   2 28.36 1.12400
## 6   2 33.54 1.02400
## 7   4 39.04 0.93320
## 8   5 45.60 0.85030
## 9   5 51.54 0.77470
```

Regression Regularization in R: glmnet

Coefficients at a given λ:

```
coef(fit, s = 0.1)
```

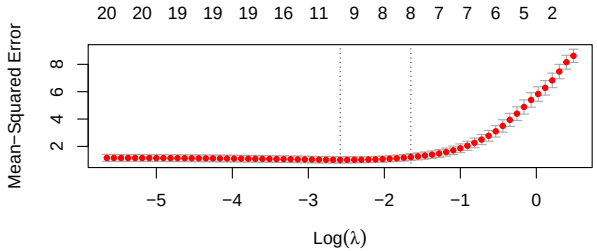
```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                       s1
## (Intercept)  0.150928072
## V1           1.320597195
## V2           .
## V3           0.675110234
## V4           .
## V5          -0.817411518
## V6           0.521436671
## V7           0.004829335
## V8           0.319415917
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Regression Regularization in R: glmnet

`glmnet` returns a whole sequence of models. Choose one by **cross-validation** with `cv.glmnet`:

```
cvfit <- cv.glmnet(x, y)
plot(cvfit)
```



Choosing λ: min vs. 1se

The two dashed lines in the CV plot:

- λ_{min}: smallest cross-validated error
- λ_{1se}: largest λ within one standard error of the minimum: a simpler model that is about as accurate (the **one-standard-error rule**)

```
c(min = cvfit$lambda.min, `1se` = cvfit$lambda.1se)
```

```
##        min        1se
## 0.07569327 0.19191003
```

Regression Regularization in R: glmnet

λ_{min} and its coefficients:

```
cvfit$lambda.min
```

```
## [1] 0.07569327
```

```
coef(cvfit, s = "lambda.min")
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept)  0.14867414
## V1           1.33377821
## V2           .
## V3           0.69787701
## V4           .
## V5          -0.83726751
## V6           0.54334327
## V7           0.02668633
## V8           0.33741131
## V9           .
## V10          .
## V11          0.17105029
## V12          .
## V13          .
## V14         -1.07552680
## V15          .
## V16          .
## V17          .
```

Example: Logistic Regression Elastic Net

For a binary response $y \in \{0, 1\}$, logistic regression models

$$P(y = 1|\mathbf{X}) = \frac{e^{\mathbf{X}\beta}}{1 + e^{X\beta}},$$

or equivalently

$$\log \frac{P(y = 1|\mathbf{X})}{P(y = 0|\mathbf{X})} = \mathbf{X}\beta,$$

the logistic (log-odds) transformation.

Example: Logistic Regression Elastic Net

Minimize:

$$\arg\min_{\beta} \left\{ -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \mathbf{x}_i' \beta - \log\left(1 + e^{\mathbf{x}_i' \beta}\right) \right] + \lambda \left(\alpha \sum_{j=1}^{p} |\beta_j| + \frac{1-\alpha}{2} \sum_{j=1}^{p} \beta_j^2 \right) \right\}$$

the logistic regression loss plus the elastic net penalty.

- Logistic regression breaks down when $p > N$ and is unstable when $N \approx p$
- The elastic net fixes this by regularizing and selecting variables

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Example: Logistic Regression Elastic Net

The glmnet binomial example:

```
library(glmnet)
data(BinomialExample)
x <- BinomialExample$x
y <- BinomialExample$y
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Example: Logistic Regression Elastic Net

Use `family = "binomial"`, and `alpha = 0.5` for an even lasso/ridge mix:

```
fit <- glmnet(x, y, family = "binomial", alpha = 0.5)
```

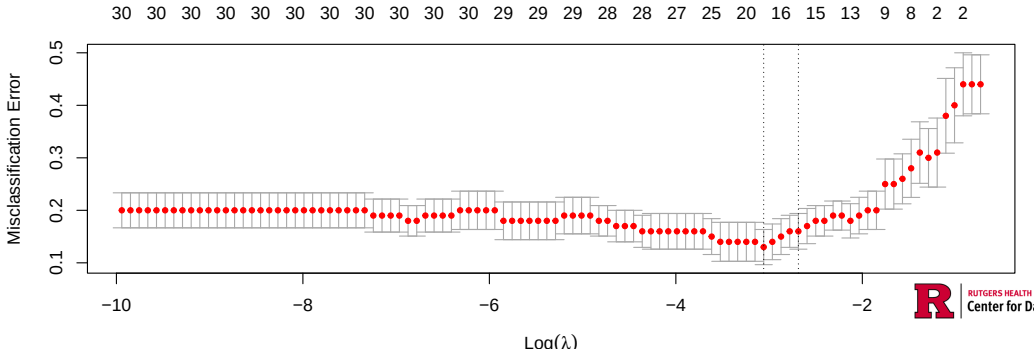
10-fold CV with misclassification error:

```
cvfit <- cv.glmnet(x, y, family = "binomial", alpha = 0.5,
                   type.measure = "class")
```

Example: Logistic Regression Elastic Net

CV results and the best λ_{min}:

```
plot(cvfit)
```



WEJ LAB
RUTGERS HEALTH
Center for Data Science

Example: Logistic Regression Elastic Net

```
coef(cvfit, s = "lambda.min")
```

```
## 31 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept)  0.24017226
## V1           .
## V2           0.35569961
## V3          -0.27959118
## V4          -0.73951856
## V5          -0.16994829
## V6          -0.53507670
## V7           .
## V8          -0.33644739
## V9           0.36808596
## V10         -0.81915175
## V11         -0.06527401
## V12         -0.01776412
## V13          .
## V14          .
## V15          .
## V16          0.12143799
## V17          .
## V18          .
## V19          .
## V20          .
## V21          .
## V22          0.15695618
## V23          0.22587034
```

WEJ LAB

RUTGERS HEALTH
Center for Data Science

Application: NHANES

National Health and Nutrition Examination Survey (CDC): interviews, physical exams, and labs on a representative sample of the US population.

- `NHANES` R package: 10,000 participants from the 2009-2012 surveys, 76 variables
- Question: who has **diabetes**? Predict it from demographics, exam, and lab values
- Many correlated predictors (BMI, weight, height; systolic and diastolic BP)

```
install.packages("NHANES")
```

NHANES: the data

```
library(NHANES); library(tidyverse)
d <- NHANES %>% filter(Age >= 20) %>% distinct(ID, .keep_all = TRUE) %>%
  select(Diabetes, Age, Gender, Race1, Education, Poverty, BMI, Weight,
         Height, Pulse, BPSysAve, BPDiaAve, TotChol, DirectChol,
         SleepHrsNight, PhysActive, Smoke100, HealthGen) %>%
  drop_na()
table(d$Diabetes)
```

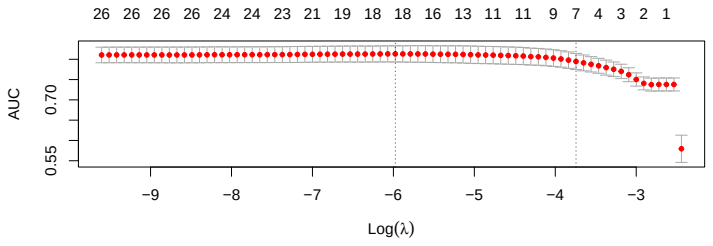
```
##
##   No  Yes
## 3151  419
```

```
X <- model.matrix(Diabetes ~ ., d)[, -1]    # dummy-code factors
y <- as.numeric(d$Diabetes == "Yes")
dim(X)
```

```
## [1] 3570   26
```

NHANES: train/test split and lasso

```
set.seed(2026)
train <- sample(nrow(X), 0.7 * nrow(X))
cv_lasso <- cv.glmnet(X[train, ], y[train], family = "binomial",
                      alpha = 1, type.measure = "auc")
plot(cv_lasso)
```



NHANES: which variables does the lasso keep?

```
b <- coef(cv_lasso, s = "lambda.1se")
round(b[b[, 1] != 0, ], 3)
```

```
##    (Intercept)            Age     Race1White            BMI     DirectChol
##         -4.747          0.036         -0.032          0.034         -0.140
## HealthGenVgood  HealthGenFair  HealthGenPoor
##         -0.079          0.302          0.877
```

Age, BMI, low HDL ("good") cholesterol, and poor self-rated health. Weight and height drop out: BMI already carries that information.

NHANES: ridge vs. elastic net vs. lasso vs. logistic

```
auc <- function(p, y) {            # Mann-Whitney form of the AUC
  r <- rank(p); n1 <- sum(y)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * sum(1 - y)) }
res <- sapply(c(ridge = 0, enet = 0.5, lasso = 1), function(a) {
  cv <- cv.glmnet(X[train, ], y[train], family = "binomial", alpha = a,
                  type.measure = "auc")
  p  <- predict(cv, X[-train, ], s = "lambda.1se", type = "response")
  c(AUC = auc(p, y[-train]), predictors = sum(coef(cv, s = "lambda.1se") != 0) - 1) })
g <- glm(y ~ ., data = data.frame(y, X)[train, ], family = binomial)
p_glm <- predict(g, data.frame(X)[-train, ], type = "response")
tab <- round(cbind(res, glm = c(auc(p_glm, y[-train]), ncol(X))), 3)
```

NHANES: ridge vs. elastic net vs. lasso vs. logistic

Test-set AUC (30% held out) and number of predictors used:

```
##                ridge  enet  lasso    glm
## AUC            0.842 0.835  0.841  0.847
## predictors 26.000 9.000 11.000 26.000
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

NHANES: what about a small study?

With 2,500 training samples, plain logistic regression does fine. Repeat with only **150** training samples, 20 times:

```
set.seed(1)
small <- replicate(20, {
  tr <- sample(train, 150)
  cv <- suppressWarnings(cv.glmnet(X[tr, ], y[tr], family = "binomial",
                                   alpha = 0.5, nfolds = 5))
  g  <- suppressWarnings(glm(y ~ ., data = data.frame(y, X)[tr, ],
                             family = binomial))
  c(enet = auc(predict(cv, X[-train, ], s = "lambda.min"), y[-train]),
    glm  = auc(predict(g, data.frame(X)[-train, ]), y[-train])) })
round(rowMeans(small), 3)
```

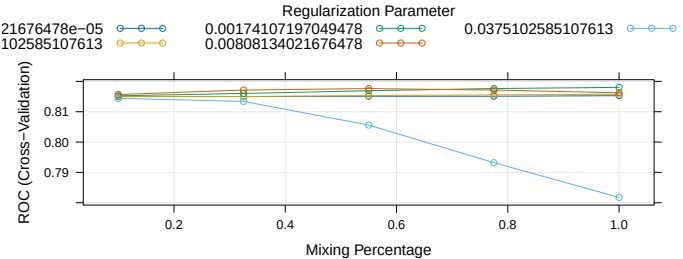
```
##  enet   glm
## 0.779 0.700
```

Tuning α and λ together with caret

`caret::train` (week 2) searches over both α and λ with cross-validation:

```
library(caret)
dtr <- data.frame(Diabetes = factor(ifelse(y == 1, "Yes", "No")), X)[train, ]
set.seed(2026)
fit_caret <- train(Diabetes ~ ., data = dtr, method = "glmnet",
                   family = "binomial", metric = "ROC", tuneLength = 5,
                   trControl = trainControl(method = "cv", number = 10,
                     classProbs = TRUE, summaryFunction = twoClassSummary))
```

Tuning α and λ together with caret



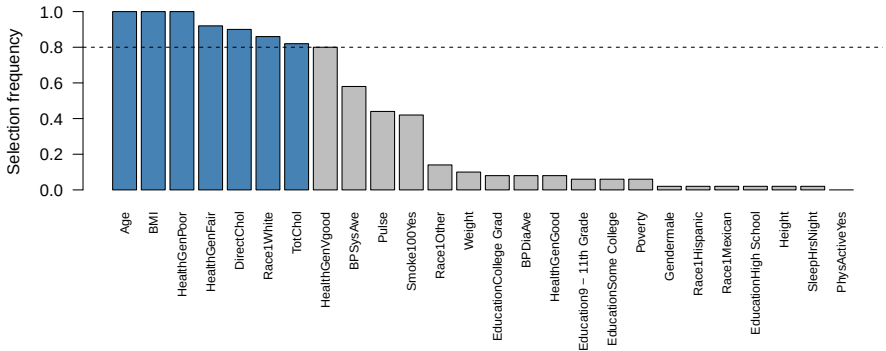
Each line is one λ. With small λ the mix barely matters; with large λ, more lasso (higher α) drops too many variables. Best: $\alpha = 1$, $\lambda = 0.0017$.

How stable is the lasso's selection?

Refit the lasso on 50 bootstrap samples and count how often each variable is kept:

```
set.seed(1)
sel <- replicate(50, {
  b  <- sample(train, replace = TRUE)
  cv <- cv.glmnet(X[b, ], y[b], family = "binomial", alpha = 1, nfolds = 5)
  as.numeric(coef(cv, s = "lambda.1se")[-1] != 0) })
freq <- sort(setNames(rowMeans(sel), colnames(X)), decreasing = TRUE)
```

How stable is the lasso's selection?



A few variables are always chosen; many come and go. Don't over-interpret a single lasso fit.

NHANES: lessons

- Large N, modest p: regularization gives about the same accuracy with **far fewer** predictors (simpler, cheaper to measure)
- Small N relative to p (150 people, ~18 cases, 26 predictors): regularization **protects** against overfitting (test AUC about 0.78 vs. 0.70)
- Selected variables are **predictive**, not causal, and can change with the sample
- Note: NHANES uses a complex survey design; for population estimates, use survey weights (`survey` package)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Summary

- Regularization adds a little bias to reduce variance and overfitting
- Ridge shrinks; lasso shrinks and selects; elastic net does both and handles correlated predictors
- Standardize predictors (glmnet does by default); don't penalize the intercept
- Tune λ (and α) by cross-validation; λ_{1se} gives a simpler model
- Large N: similar accuracy with fewer predictors. Small N: better accuracy
- Selected variables are predictive, not causal

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] caret_6.0-94    lattice_0.22-5  lubridate_1.9.3 forcats_1.0.0
##  [5] stringr_1.5.1   dplyr_1.1.4     purrr_1.0.2     readr_2.1.5
##  [9] tidyr_1.3.1     tibble_3.2.1    ggplot2_3.4.4   tidyverse_2.0.0
## [13] NHANES_2.1.4    glmnet_4.1-8    Matrix_1.6-5
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.4         shape_1.4.6          xfun_0.41
##  [4] recipes_1.0.9        tzdb_0.4.0           vctrs_0.6.5
```